FIT3155: Solutions to conceptual questions in Week 3 Lab sheet

1. For the string $S =$ $\overset{1}{\texttt{m}}\ \overset{2}{\texttt{i}}\ \overset{3}{\texttt{s}}\ \overset{4}{\texttt{s}}\ \overset{5}{\texttt{i}}\ \overset{6}{\texttt{s}}\ \overset{7}{\texttt{s}}\ \overset{8}{\texttt{i}}\ \overset{9}{\texttt{p}}\ \overset{10}{\texttt{p}}\ \overset{11}{\texttt{i}}\ \$$, where $\$$ is a special terminal character that is assumed to be lexicographically smaller than any other character in S.

   (a) On paper, construct its sorted cyclic permutation matrix M.

   Ans. 1(a):

```
SA  |      M
----|---------------
 12 | $mississippi
 11 | i$mississipp
  8 | ippi$mississ
  5 | issippi$miss
  2 | ississippi$m
  1 | mississippi$
 10 | pi$mississip
  9 | ppi$mississi
  7 | sippi$missis
  4 | sissippi$mis
  6 | ssippi$missi
  3 | ssissippi$mi
```

   (b) Identify the BWT of S from M and reason the worst-case time complexity of constructing a BWT using this approach.

   Ans. 1(b):
   BWT = `ipssm$pissii`

   For any string $S[1 \ldots n]$ (including the \$ terminal character), generating an unsorted matrix M' of all cyclic rotations of S requires $O(n^2)$ effort (since each row of M' requires $O(n)$ time to generate). Sorting rows of M' using a comparison-based sort to generate the matrix M of sorted cyclic rotations takes $O(n^2 \log n)$ time, since there are n strings to sort involving at most $n \log n$ comparisons in the worst-case, but, importantly, each comparison requires $O(n)$ effort in the worst-case as two strings of length n have to be compared.

(c) On paper, construct its suffix array.* Verify that the information stored in the suffix array correspond to the ordering of rows in M. Reason why this hold in general.

Ans. 1(c):

Suffix Array (SA) shown in the first column of Ans 1(a).

Reasoning: Each row $M[i]$ (which is a cyclic rotation of S) can be decomposed into a form $\alpha_i\$\beta_i$, wherre α_i is the substring/prefix from the start of the row up to \$ (exclusive) and β_i is the substring/suffix from after \$ till the end of the row.

Without loss of generality, consider any pair of rows of M such that

$$M[i] \prec M[j] \implies \alpha_i\$\beta_i \prec \alpha_j\$\beta_j.$$

Note, $|\alpha_i| \neq |\alpha_j|$ (i.e., their lengths can never be equal). This is a consequence of each column of M having exactly one \$ character.

- If $|\alpha_i| > |\alpha_j|$, then the $|\alpha_j|$-sized prefix of α_i must be lexicographically smaller than α_j: $\alpha_i[1 \dots |\alpha_j|] \prec \alpha_j$. Why? If that prefix were equal to α_j in the worst case, then at index $|\alpha_j| + 1$, the character in $M[j]$ is \$ while the character in $M[i]$ is some character $c \neq \$$. Since $\$ \prec c$, then this would imply $M[j] \prec M[i]$ which contradicts our assumption that row i came before row j. Hence,

  $$M[i] \prec M[j] \implies \alpha_i[1 \dots |\alpha_j|] \prec \alpha_j \implies \alpha_i \prec \alpha_j.$$

- If $|\alpha_i| < |\alpha_j|$, then α_i must either be a prefix of α_j or lexicographically smaller than the $|\alpha_i|$-sized prefix of α_j:

  $$\alpha_i \preceq \alpha_j[1 \dots |\alpha_i|].$$

  If $\alpha_i = \alpha_j[1 \dots |\alpha_i|]$, then at position $|\alpha_i| + 1$, $M[i]$ has \$ while $M[j]$ has a character $c \neq \$$. Since $\$ \prec c$, this was the reason why $M[i] \prec M[j]$. Otherwise, there exists a smallest $1 \leq k \leq |\alpha_i|$ where $\alpha_i[k] \neq \alpha_j[k]$ and since $M[i] \prec M[j]$, it must be that $\alpha_i[k] \prec \alpha_j[k]$, implying $\alpha_i \prec \alpha_j[1 \dots |\alpha_i|] \implies \alpha_i \prec \alpha_j$.

Since this holds true for any pair of rows where $M[i] \prec M[j]$, this implies the order of rows in M implies the sorted order of the string's suffixes: $\{\alpha_1\$ \prec \alpha_2\$ \prec \dots \prec \alpha_n\$\}$.

(d) Confirm visually that each **column** of M is a permutation of characters in S, and explain why this holds for any string.

Ans. 1(d):

Let M' be the **unsorted** matrix of all cyclic rotations of S. Each row of M' is obtained by cyclically shifting the characters of S by some $0 \leq k < n$ positions. For any column j, the character in row i is given by (using 1-based indexing):

$$M'[i][j] = S[((i + j - 2) \bmod n) + 1].$$

For any fixed column j and varying $i \in [1, \dots, n]$, the expression $((i+j-2) \bmod n)+1$ will take on every value in the range $[1, \dots, n\}$ exactly once. Hence, column j contains every character of S exactly once, i.e., it is a permutation of the characters of S.

Since M is obtained by sorting the rows of M', which only permutes the rows, each column of M remains a permutation of the characters of $S[1 \dots n]$.

(e) How can the BWT be computed from the information in the suffix array? Assume the you have a method to compute the suffix array of any given string. What is the time complexity of computing the BWT of a string given its suffix array?

Ans. 1(e):

For a string $S[1 \dots n]$, constructing its suffix array by explicitly sorting all suffixes takes $O(n^2 \log n)$ time in the worst case.

Given the suffix array $SA[1 \dots n]$, the Burrows–Wheeler Transform (BWT) can be computed from it in $O(n)$ time as it involves the following computation for each position p

*A suffix array is an array storing positions of all suffixes of a string in a sorted order.

stored in the suffix array (accessing them the order captured by SA):

$$\forall 1 \le i \le n,\ p = SA[i] \implies \text{BWT}[i] = \begin{cases} S[n] & \text{if } p = 1, \\ S[p-1] & \text{otherwise.} \end{cases}$$

(Alternately, using modular arithmetic (while accounting for 1-based indexing), we have $\text{BWT}[i] = S[((p-2) \bmod n) + 1]$.)

(f) For each character $L[i]$ in the last column (L) of M, visually identify its corresponding position p in the first column (F) of M. (This is performing a visual LF-mapping on this example string.)

Ans. 1(f):

```
|  p  |   M         | L[i] --> F[p]
|-----|-------------|---------------
| 1   |$mississippi| L[1] --> F[2]
| 2   |i$mississipp| L[2] --> F[7]
| 3   |ippi$mississ| L[3] --> F[9]
| 4   |issippi$miss| L[4] --> F[10]
| 5   |ississippi$m| L[5] --> F[6]
| 6   |mississippi$| L[6] --> F[1]
| 7   |pi$mississip| L[7] --> F[8]
| 8   |ppi$mississi| L[8] --> F[3]
| 9   |sippi$missis| L[9] --> F[11]
|10   |sissippi$mis| L[10]--> F[12]
|11   |ssippi$missi| L[11]--> F[4]
|12   |ssissippi$mi| L[12]--> F[5]
```

(g) For any $L[i] \mapsto F[p]$, verify that $L[p]$ immediately precedes $L[i]$ in S, and explain why this holds in general.

Ans. 1(g): M is constructed such that any $1 \le k \le n$ successive columns contain all k-mers of the string $S[1 \dots n]$ in some permuted order. Therefore, cyclically, characters in column L precede the corresponding character in column F in the original string $S[1 \dots n]$.

2. Prove the following statement involving LF-mapping: If $L[i] \mapsto F[p]$, then $p = \texttt{rank}(L[i]) + \texttt{nOccurrences}(L[i] \in L[1 \dots i))$[†]

Ans. 2: See proof document on Moodle

3. BWT of some string S is `oooloooooolwml$`. What is S?

Ans. 3:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14
o o o l o o o o o l  w  m  l  $
```

```
S is inverted backwards while mapping some $L[i]--->F[pos]$ and repeating the process with i=pos
Start with i=1. L[1] = o = S[13] (since S[14]= F[1] = $ is always at i = 1 in the suffix array,
                                                    its preceding char S[13] has to be L[1]):
                                                                          L[ 1] = o = S[13]
i =   1     L[ 1] = o   => pos =  Rank(o) + nOcc(o, L[1... 1)) =  6 +  0 =  6   L[ 6] = o = S[12]
i =   6     L[ 6] = o   => pos =  Rank(o) + nOcc(o, L[1... 6)) =  6 +  4 = 10   L[10] = l = S[11]
i =  10     L[10] = l   => pos =  Rank(l) + nOcc(l, L[1...10)) =  2 +  1 =  3   L[ 3] = o = S[10]
i =   3     L[ 3] = o   => pos =  Rank(o) + nOcc(o, L[1... 3)) =  6 +  2 =  8   L[ 8] = o = S[ 9]
i =   8     L[ 8] = o   => pos =  Rank(o) + nOcc(o, L[1... 8)) =  6 +  6 = 12   L[12] = m = S[ 8]
i =  12     L[12] = m   => pos =  Rank(m) + nOcc(m, L[1...12)) =  5 +  0 =  5   L[ 5] = o = S[ 7]
i =   5     L[ 5] = o   => pos =  Rank(o) + nOcc(o, L[1... 5)) =  6 +  3 =  9   L[ 9] = o = S[ 6]
i =   9     L[ 9] = o   => pos =  Rank(o) + nOcc(o, L[1... 9)) =  6 +  7 = 13   L[13] = l = S[ 5]
i =  13     L[13] = l   => pos =  Rank(l) + nOcc(l, L[1...13)) =  2 +  2 =  4   L[ 4] = l = S[ 4]
i =   4     L[ 4] = l   => pos =  Rank(l) + nOcc(l, L[1... 4)) =  2 +  0 =  2   L[ 2] = o = S[ 3]
i =   2     L[ 2] = o   => pos =  Rank(o) + nOcc(o, L[1... 2)) =  6 +  1 =  7   L[ 7] = o = S[ 2]
i =   7     L[ 7] = o   => pos =  Rank(o) + nOcc(o, L[1... 7)) =  6 +  5 = 11   L[11] = w = S[ 1]
The inversion can stop when the whole string S[1...n] is recovered.
inverseBWT(oooloooooolwml$) = woolloomooloo$
```

[†]Range L[1...i) is exclusive of i.

```
Sanity-check. Running one more time recovers $, and will cycle through the same inversion if continued...
i =  11     L[11] = w   => pos =  Rank(w) + nOcc(w, L[1...11)) = 14 +  0 = 14   l[11]  = $ = S[14]
...
```

$S =$ 1 2 3 4 5 6 7 8 9 10 11 12 13
w o o l l o o m o o l o o $.

4. In general, what is the worst-case time and space complexity of inverting a BWT **without** preprocessing the `rank` and `nOccurrences` functions.

Ans. 4:

Computing rank for any given character $L[i]$, requires counting the number of times one has seen any character in $L[1 \dots n]$ that is smaller than $L[i]$, taking $O(1)$ (auxiliary) space and $O(n)$ effort.

Computing nOccurrences for any given character $L[i]$ before it, requires counting for $L[i] \in L[1 \dots i)$, taking $O(1)$ (auxiliary) space and $O(n)$ effort in the worst-case.

If running inversion without preprocessing, these two functions are called n times each to generate the original string of size n, requiring $O(n)$ (auxiliary) space and $O(n^2)$ effort.

5. Write pseudocode to preprocess a `rank` data structure. Characterize the worst-case time and space complexity of its construction.

Ans. 5:

```
%Step 1: Initialize counts and rank array
Initialize c[1...|Σ|] = r[1...|Σ|] = {0, 0, ..., 0}

%Step 2: scan L[1...n] and count each observed char
loop i from 1 to n
    increment c[index(L[i])]

%Step 3: convert counts into rank
r[1] ← 1
loop i from 2 to |Σ|
    r[i] ← r[i − 1] + c[i − 1]
```

From the above pseudocode, space is clearly bounded by $O(|\Sigma|)$ to hold the counts and rank. Effort/Time is bounded by $O(n)$ in step 2 and $O(|\Sigma|)$ in step 3, so $O(n + |\Sigma|)$ overall.

6. Write pseudocode to preprocess a `nOccurrences` data structure, assuming you would use it only for inversion. Characterize the worst-case time and space complexity of its construction.

Ans. 6:

```
%Step 1: Initialize nOcc array of size n storing nOcc(L[i] ∈ L[1...i))...
Initialize nOcc[1...n] = {0, 0, ..., 0}
%         ... and runC (running counts) array of size |Σ|
Initialize runC[1...|Σ|] = {0, ..., 0}

%Step 2: Scan L[1...n], assign nOcc based on running counts.
loop i from 1 to n
    assign nOcc[i] ← runC[index(L[i])]
    increment runC[index(L[i])]
```

From the above pseudocode, space is clearly bounded by $O(n + |\Sigma|)$ to hold the final number of occurrences before each position i and the auxiliary array of size $|\Sigma|$ to help its accumulation. Effort/Time is bounded by $O(n + |\Sigma|)$ in step 1 and $O(n)$ in step 2.

7. When BWT is inverted into its original string using preprocessed `rank` and `nOccurrences` data structures, what would be the worst-case space and time complexity of inversion (including the preprocessing steps).

Ans. 7:

Inversion loop needs to run at most n times over all characters of L (in some order), with recovery of each character in the original string requiring $O(1)$ space and time, by using preprocessed values from rank and nOcc data structures.

Worst-case space and time complexities for one-time preprocessing of rank are $O(|\Sigma|)$ and $O(n + |\Sigma|)$ respectively.

Worst-case space and time complexities for one-time preprocessing of nOcc (for **inversion** purpose only) are both $O(n + |\Sigma|)$.

So, inversion with preprocessing of the above two data structures takes worst-case $O(n+|\Sigma|)$ time and space.

8. For `pat`$[1 \ldots 3]$ = `iss`.

   (a) Work out step-by-step the iterative backward search of the pattern using the BWT you have constructed for Q1 as a search index. In your working on paper, mainly identify the updates to the start-position (sp) and end-position (ep).

Ans. 8(a):

Pattern matching using BWT as index is always in the reverse order of the characters in pat = i s s :

```
Initialize sp = 1 and ep = n

SA  || p |   M        |
----||---|------------|
 12 || 1 |$mississippi| <--- sp
 11 || 2 |i$mississipp|
  8 || 3 |ippi$mississ|
  5 || 4 |issippi$miss|
  2 || 5 |ississippi$m|
  1 || 6 |mississippi$|
 10 || 7 |pi$mississip|
  9 || 8 |ppi$mississi|
  7 || 9 |sippi$missis|
  4 || 0 |sissippi$mis|
  6 || 1 |ssippi$missi|
  3 || 2 |ssissippi$mi| <--- ep
----------------------------------------------------------------------
Iteration 1: search for pat[3] = s:
----------------------------------------------------------------------
sp (updated) =  Rank(s) + nOcc(s, L[1... 1))     =  9 +  0    = 9
ep (updated) =  Rank(s) + nOcc(s, L[1... 12]) -1 =  9 +  4 -1 = 12
SA  || p |   M        |
----||---|------------|
 12 || 1 |$mississippi|
 11 || 2 |i$mississipp|
  8 || 3 |ippi$mississ|
  5 || 4 |issippi$miss|
  2 || 5 |ississippi$m|
  1 || 6 |mississippi$|
 10 || 7 |pi$mississip|
  9 || 8 |ppi$mississi|
  7 || 9 |sippi$missis| <--- sp
  4 ||10 |sissippi$mis|
  6 ||11 |ssippi$missi|
  3 ||12 |ssissippi$mi| <--- ep
----------------------------------------------------------------------
```

```
Iteration 2: search for pat[2] = s:
-----------------------------------------------------------------
sp (updated) =  Rank(s) + nOcc(s, L[1... 9))     =  9 +  2    = 11
ep (updated) =  Rank(s) + nOcc(s, L[1... 12]) -1 =  9 +  4 -1 = 12
SA  || p |   M         |
----||---|------------|
 12 || 1 |$mississippi|
 11 || 2 |i$mississipp|
  8 || 3 |ippi$mississ|
  5 || 4 |issippi$miss|
  2 || 5 |ississippi$m|
  1 || 6 |mississippi$|
 10 || 7 |pi$mississip|
  9 || 8 |ppi$mississi|
  7 || 9 |sippi$missis|
  4 || 0 |sissippi$mis|
  6 || 1 |ssippi$missi| <--- sp
  3 || 2 |ssissippi$mi| <--- ep
-----------------------------------------------------------------
Iteration 3: search for pat[1] = i:
-----------------------------------------------------------------
sp (updated) =  Rank(i) + nOcc(s, L[1... 11))     =  2 +  2    = 4
ep (updated) =  Rank(i) + nOcc(s, L[1... 12]) -1  =  2 +  4 -1 = 5
SA  || p |   M         |
----||---|------------|
 12 || 1 |$mississippi|
 11 || 2 |i$mississipp|
  8 || 3 |ippi$mississ|
  5 || 4 |issippi$miss| <--- sp
  2 || 5 |ississippi$m| <--- ep
  1 || 6 |mississippi$|
 10 || 7 |pi$mississip|
  9 || 8 |ppi$mississi|
  7 || 9 |sippi$missis|
  4 || 0 |sissippi$mis|
  6 || 1 |ssippi$missi|
  3 || 2 |ssissippi$mi|
```

Note: As long as $ep \geq sp$, after each iteration, the range $SA[sp, ep]$ in general (or, in this example, $SA[4, 5] = \{5, 2\}$) gives the loci/positions in the original string (in this example, `1 2 3 4 5 6 7 8 9 10 11 12` / `m i s s i s s i p p i $`) where the growing suffix of the pattern appears.

(b) Understand what the updated range $L[sp \ldots ep]$ is conveying after each iteration of the backward search.

Ans. 8(b):

For a pattern $\texttt{pat}[1 \ldots m]$, consider any k-th iteration of the backward search using the BWT as a search index, where $1 \leq k \leq m$. At this point, say, we have already searched for the suffix $\alpha = \texttt{pat}[m - k + 2 \ldots m]$ of length $k - 1$. At the start of this k-th iteration, any interval $[sp, ep]$ corresponds to the range of rows of M where the first $k - 1$ columns will all contain the substring α. (This is same as $SA[sp \ldots ep]$ containing loci of all suffixes of $S[1 \ldots n]$ that are prefixed by α.)

The corresponding region $L[sp \ldots ep]$ contains exactly those characters in the reference text, that immediately precede occurrences of α. Thus, the characters in the updated range $L[sp \ldots ep]$ represent all possible leftward-extensions of α possible in the reference text that one is searching (using the text's BWT as an index).

(c) At the end of the search, how would you work out the **number** of occurrences of the pattern in the original string?

Ans. 8(c):

Number of occurrences of the pattern $= ep - sp + 1$.

For this example (8(a)):

$5 - 4 + 1 = 2$ occurrences of the pattern `1 2 3` / `i s s` in the text `1 2 3 4 5 6 7 8 9 10 11 12` / `m i s s i s s i p p i $` .

(d) What additional information do you need to be able to identify the **loci** of occurrences of the pattern in the original string? **Ans. 8(d):**

One would need $SA[1 \ldots n]$ in addition to $L[1 \ldots n]$ to be able to identify the loci (as covered above – see note at the end of Ans 8(a)).

9. Prove the correctness of the update rules for start and end positions duing the backwards search:

   - $sp_{\text{new}} = \texttt{rank}(x) + \texttt{nOccurrences}(x \in L[1 \ldots sp))$
   - $ep_{\text{new}} = \texttt{rank}(x) + \texttt{nOccurrences}(x \in L[1 \ldots ep] - 1)$

   Ans. 9:

   See proof document on moodle.

10. Write pseudocode to preprocess a `nOccurrences` data structure for its use in pattern matching. Characterize the worst-case time and space complexity of its construction.
    Ans. 10:

```
1  %Step 1: Initialize nOcc Table of size n × |Σ| storing
2  %       nOcc(x ∈ L[1...i)), ∀x ∈ Σ.
3  Initialize nOcc[1...n][1...|Σ|] = {{0,0,...,0},...,{0,0,...,0}}
4  %         ... and runC (running counts) array of size |Σ|
5  Initialize runC[1...|Σ|] = {0,...,0}
6
7  %Step 2: scan L[1...n], assign nOcc Table based of running counts
8  loop i from 1 to n
9     assign (table row) nOcc[i] ← runC[1...|Σ|]
10    increment runC[index(L[i])]
```

    (Note, the above nOcc **table** data structure stores the number of occurrences of any possible character in the alphabetin the range $L[1 \ldots i)$ (i.e., exclusive of index i). Figure out how one could use this (exclusive) definition to be able to also use it to get an $O(1)$-time answer (with no additional space), for any query for the number of occurrences of any charcter in the alphabet, inclusive of i, necessary for updates to ep during pattern matching!)

    From the above pseudocode, space and time are clearly bounded by $O(n \times |\Sigma|)$.

11. Characterize the worst-case time and space complexity to search for a pattern $\text{pat}[1 \ldots m]$ using the BWT as the search index.

    Ans. 11:

    Preprocessing the rank and nOcc tables takes $O(n \cdot |\Sigma|)$ time and space in the worst case. This is a one-time preprocessing step and does not need to be repeated for each pattern search.

    Backward search for a pattern of length m requires m iterations taking constant effort and space to carry out each iteration using the precomputed data structures.

    Computing the number of occurrences (i.e., the size of the final SA interval) takes $O(1)$ time.

    Reporting all occurrences (the positions in the text) requires $O(\text{multiplicity})$ time, where multiplicity is the number of matches of the pattern in the text.

    Overall, after preprocessing, the pattern matching phase runs in $O(m + \text{multiplicity})$ time using $O(n \times |\Sigma|)$ space.

-=0=-
END
-=0=-